



UNIVERSITATEA NAȚIONALĂ DE ȘTIINȚE ȘI TEHNOLOGIE
POLITEHNICA BUCUREȘTI
FACULTATEA DE ȘTIINȚE APLICATE

Raport de Practică 2026

Analiza și modelarea consumului de combustibil în regim de trafic mixt

Student: **Darko TRIFAN**

Tutore practică: **Simona Mihaela BIBIC**

Cuprins

1	Sinteză	3
2	Introducere	4
2.1	Motivația alegerii temei-Problematika	4
2.2	Obiectivele lucrării	5
3	Factorii care influențează consumul de combustibil	6
3.1	Specificațiile tehnice ale vehiculului	6
3.2	Dinamica traseului și a infrastructurii	6
3.3	Regimul de trafic dinamic	7
4	Modelul matematic și teoretizarea consumului	8
4.1	Fizica mișcării vehiculului	8
4.2	Determinarea ecuațiilor de consum	9
4.3	Modelarea factorului de trafic	9
5	Arhitectura sistemului și implementarea numerică	10
5.1	Structura datelor utilizate	10
5.2	Logica algoritmului de calcul	11
5.3	Biblioteci utilizate în Python	13
5.4	Descrierea algoritmilor și o modulelor de calcul	14
6	Dezvoltarea interfeței grafice și arhitectura aplicației	17
6.1	Cadrul Streamlit și modelul de execuție	17
6.2	Modulele interactive de vizualizare	17
6.3	Structura bazei de date auto	18
7	Studiu de caz și rezultate experimentale	18
7.1	Definirea scenariilor de test	18
7.2	Compararea rezultatelor pe tipuri de motorizări	18
7.3	Interpretarea grafică a datelor	19
8	Resurse	19
9	Concluzii și perspective de dezvoltare?	20

1 Sinteză

Prezenta lucrare propune o analiză a impactului variabilelor de trafic și al specificațiilor tehnice ale vehiculelor asupra consumului de carburant. În cadrul acestui proiect, este dezvoltat un model de calcul care integrează caracteristicile constructive ale motorului (masă, capacitate cilindrică, combustibil) cu dinamica traseului și indicii de aglomerație specifici diferitelor momente ale zilei. Prin împărțirea traseului în segmente și atribuirea unor viteze de deplasare corelate cu intervalele orare de vârf, modelul simulează comportamentul vehiculului în regim Stop-and-Go. Rezultatele obținute în urma studiilor de caz demonstrează modul în care optimizarea rutei și planificarea temporală a deplasării pot contribui semnificativ la reducerea consumului total. La baza modelului matematic stă ecuația forțelor opozante care acționează asupra mașinii. Pentru a aduce totul în cadrul traficului real, am împărțit traseul pe segmente de trafic și am aplicat un Factor de Congestie, care crește consumul teoretic în funcție de cât de mult scade viteza medie din cauza aglomerației și a timpilor de așteptare la semafor. Din punct de vedere practic, am implementat acest model într-o aplicație scrisă în Python, folosind o structură orientată pe obiecte (cu clase pentru vehicul, segment și trafic). Aplicația se conectează la API-uri de trafic în timp real (cum ar fi Waze, OSRM și Nominatim) și afișează datele pe o hartă interactivă realizată în Streamlit, Folium și Plotly. Prin scenarii simulate pe același traseu la ore diferite (vârf, zi și noapte), se demonstrează că o planificare mai bună a orei de plecare și a rutei poate reduce considerabil costurile legate de combustibil. Astfel, este oferit un instrument util și precis pentru calculul cheltuielilor.

de pus sub forma de subsection

Principalele elemente de originalitate ale aplicației sunt definite în cele ce urmează:

- Ceea ce aduce nou și original acest proiect este că permite personalizarea calculului pentru fiecare utilizator în parte, astfel fiind construit un algoritm care preia toate datele tehnice reale ale vehiculului (cilindree, tip de motorizare, transmisie) și le aplică direct pe ecuațiile dinamicii din trafic.
- Abordarea propusă face trecerea de la evaluarea teoretică la un instrument practic de decizie: algoritmul nu doar că estimează consumul instantaneu și cel total pe diferite segmente de drum, dar și transpune aceste date în costuri financiare directe și timp real de parcurs. Prin integrarea datelor dinamice de trafic furnizate în timp real de API-uri externe, modelul se adaptează instantaneu con-

dițiilor variabile din șosea, oferind utilizatorului posibilitatea de a compara pe loc eficiența economică a diferitelor opțiuni de rulare sau intervale orare.

- Interfața este construită pe o arhitectură modulară ce permite navigarea prin tab-uri generate dinamic. Aportul tehnic principal constă în fuziunea datelor de rutare: sistemul combină informațiile de trafic în timp real și întârzierile extrase direct din rețeaua Waze cu geometria de înaltă rezoluție oferită de serviciul OSRM.

2 Introducere

2.1 Motivația alegerii temei-Problematica

Contextul socio-economic actual este puternic marcat de o urbanizare accelerată și de o creștere explozivă a industriei auto. Motivația principală în alegerea acestei teme este necesitatea acută de a optimiza consumul de carburant, transformând deplasările rutiere într-un proces cât mai eficient din punct de vedere economic. Într-un context în care prețurile combustibililor cunosc fluctuații constante și reprezintă o pondere semnificativă atât în bugetele personale ale conducătorilor auto, cât și în costurile operaționale ale companiilor de logistică, minimizarea acestor cheltuieli a devenit o prioritate absolută. Deși producătorii auto investesc masiv în dezvoltarea unor motoare cu un randament teoretic tot mai bun, eficiența reală și costul efectiv per kilometru sunt dictate, în ultimă instanță, de condițiile de drum și de traficul dinamic. Rularea în regim de aglomerație urbană, caracterizată prin opriri și accelerări repetate, anulează frecvent avantajele tehnologice ale vehiculului, generând pierderi financiare substanțiale care pot fi evitate. Astfel, am considerat esențială dezvoltarea unui model de calcul și analiză care să demonstreze că obținerea unui cost cât mai redus nu ține doar de specificațiile mașinii, ci și de o bună planificare a rutei și a timpului de deplasare. Proiectul meu își propune oferirea unei soluții practice pentru a minimiza cheltuielile în ceea ce privește combustibilul, arătând că deciziile informate pot conduce la economii financiare reale pe orice traseu.

Problematică vs Soluții

- PROBLEMA IDENTIFICATĂ: Imposibilitatea anticipării precise a consumului real de combustibil în regim de trafic intens pe baza metodelor statice standard.

- SOLUȚIA PROPUȘĂ: Dezvoltarea unui model matematic și implementarea unui algoritm în Python capabil să estimeze timpii și costurile în mod dinamic, aplicând factorul de congestie.

2.2 Obiectivele lucrării

Scopul principal al acestui proiect de practică constă în proiectarea și implementarea unui model numeric riguros, cât și un algoritm în Python, pentru estimarea și analiza consumului de carburant, adaptat la specificul traficului dinamic, având ca obiectiv final optimizarea bugetului și obținerea unui cost de deplasare cât mai scăzut. Pentru a atinge acest scop, am structurat obiectivele specifice ale lucrării, alături de rezultatele și metodele asociate, în tabelul de mai jos:

Nr.	Obiectiv specific	Metodologie / Instrumente	Rezultatul așteptat
O1	Identificarea și formalizarea matematică a factorilor care influențează direct consumul și costurile.	Ecuatiile dinamicii vehiculului, modelarea forțelor opozante (F_{rul} , F_{aer}).	Set de ecuații matematice calibrate pentru modelare.
O2	Proiectarea și scrierea algoritmului de calcul bazat pe segmente de drum și intervale orare.	Programare orientată pe obiecte (OOP) în Python , modelarea claselor Vehicul și Traseu .	Script Python pentru simularea consumului real și a costurilor.
O3	Simularea scenariilor pe același traseu la ore diferite (vârf vs. liber).	Generarea și analiza profilelor de viteză și a timpilor de tranzit în Python.	Demonstrarea matematică și numerică a reducerii costurilor prin planificare orară.
O4	Evaluarea comparativă a două motorizări diferite (subcompactă vs. SUV).	Rularea simulărilor în Python cu seturi de date specifice pe rute identice.	Identificarea configurației optime din punct de vedere al raportului cost-eficiență.

3 Factorii care influențează consumul de combustibil

3.1 Specificațiile tehnice ale vehiculului

Consumul de combustibil este, în prima instanță, o consecință directă a modului în care energia chimică stocată în carburant este convertită în energie mecanică de către ansamblul motopropulsor. Parametrii constructivi ai vehiculului definesc pragul minim de energie necesar deplasării:

- **Masa vehiculului:** Influențează în mod direct forța de inerție. Cu cât masa este mai mare, cu atât lucrul mecanic necesar pentru accelerare (frecvent în mediul urban) crește substanțial.
- **Capacitatea cilindrică și arhitectura motorului:** Motoarele cu o cilindree mare oferă un cuplu generos, însă prezintă pierderi prin frecare internă mai mari la sarcini parțiale. Motoarele moderne, supraalimentate, optimizează acest aspect, dar sensibilitatea lor la variațiile de sarcină din trafic rămâne ridicată.
- **Tipul de combustibil:** Motoarele diesel funcționează pe baza unui ciclu termodinamic cu un randament intrinsec mai ridicat comparativ cu cele pe benzină, oferind un consum mai redus. Sistemele hibride aduc un avantaj major în traficul urban prin recuperarea energiei la frânare.
- **Tipul transmisiei:** Transmisiile manuale permit un control direct asupra turației, dar eficiența lor depinde de stilul conducătorului. Transmisiile automate moderne (cu dublu ambreiaj) optimizează punctele de schimbare a treptelor.

3.2 Dinamica traseului și a infrastructurii

Mediul fizic în care se realizează deplasarea impune constrângeri severe asupra modului de funcționare a motorului. Topologia traseului determină profilul de sarcină aplicat vehiculului:

- **Profilul geometric (rampe și pante):** Deplasarea pe un plan înclinat (rampă) poate dubla sau tripla consumul instantaneu. Coborârea pantelor permite utilizarea frânei de motor, compensând parțial consumul.
- **Configurația drumului:** Segmentele urbane cu intersecții frecvente, sensuri giratorii și treceri de pietoni impun cicluri repetate de decelerare și accelerare. Autostrăzile permit o circulație cu viteză constantă, ceea ce duce la o stabilizare a consumului de combustibil, fără fluctuații.

3.3 Regimul de trafic dinamic

Traficul este puternic dependent de factorul temporal și social. Dinamica traficului alterează fundamental modul de rulare al mașinii:

- **Indicele de aglomerație (Congestion Index):** Măsoară întârzierile și densitatea traficului dintr-o zonă urbană, comparând timpul de călătorie în trafic intens cu cel dintr-o perioadă fără trafic (liberă). Un indice ridicat este asociat cu apariția regimului Stop-and-Go, în care vehiculul petrece perioade lungi funcționând la ralanti sau accelerând de pe loc în trepte inferioare de viteză, regim în care randamentul termic al motorului este minim.
- **Intervalele orare:** Modelele de trafic prezintă caracteristici ciclice bine definite. Perioadele de vârf (dimineața în intervalul 07:30-09:30 și după-amiaza în intervalul 16:30-18:30) se caracterizează prin congestii severe pe arterele principale, în timp ce intervalele nocturne oferă un regim de rulare constant, ideal pentru minimizarea consumului.

4 Modelul matematic și teoretizarea consumului

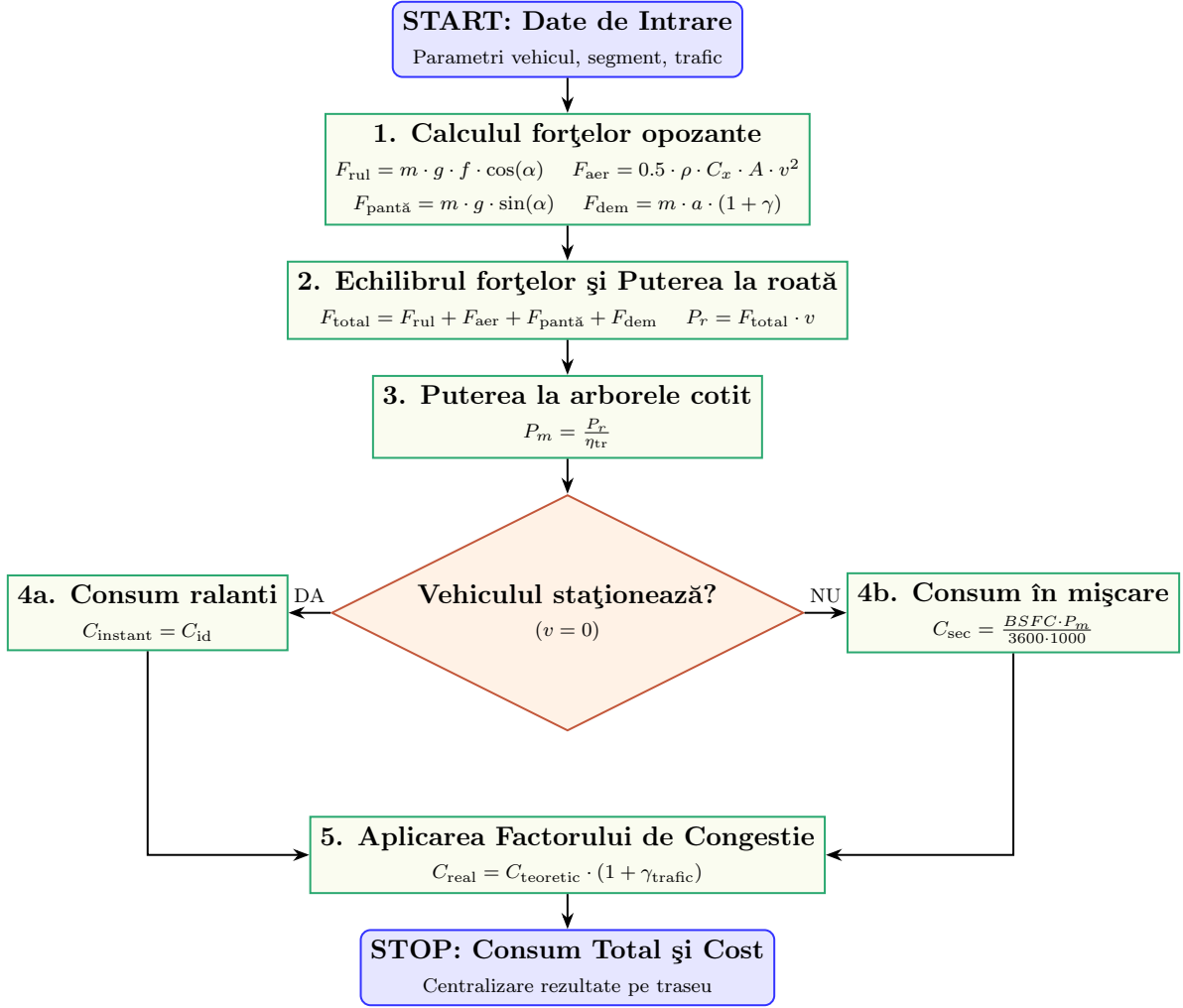


Figura 1: Schema logică a fluxului matematic pentru modelarea consumului de combustibil.

4.1 Fizica mișcării vehiculului

Pentru a modela precis consumul, este necesar să calculăm puterea totală la roată (P_r) solicitată de vehicul pentru a învinge forțele opozante. Ecuația fundamentală a dinamicii autovehiculului descrie echilibrul forțelor:

$$F_{total} = F_{rul} + F_{aer} + F_{pantă} + F_{dem}$$

- **Rezistența la rulare (F_{rul}):** Generată de deformarea pneului pe suprafața de rulare. Se calculează ca: $F_{rul} = m \cdot g \cdot f \cdot \cos(\alpha)$, unde m este masa, g este accelerația gravitațională, f este coeficientul de rulare, iar α este unghiul pantei.

- **Rezistența aerodinamică (F_{aer}):** Devine predominantă la viteze mari. Formula sa este: $F_{\text{aer}} = 0.5 \cdot \rho \cdot C_x \cdot A \cdot v^2$, unde ρ este densitatea aerului, C_x este coeficientul aerodinamic, A este aria secțiunii transversale, iar v este viteza vehiculului.
- **Rezistența la pantă ($F_{\text{pantă}}$):** Reprezintă componenta gravitațională:

$$F_{\text{pantă}} = m \cdot g \cdot \sin(\alpha).$$
- **Forța de demaraj (F_{dem}):** Necesară pentru accelerare: $F_{\text{dem}} = m \cdot a \cdot (1 + \gamma)$, unde a este accelerația, iar γ este coeficientul maselor rotative în mișcare de rotație.

4.2 Determinarea ecuațiilor de consum

- Puterea necesară la arborele cotit al motorului, P_m , se obține prin aplicarea randamentului transmisiei η_{tr} :

$$P_m = \frac{P_r}{\eta_{\text{tr}}}$$

- Consumul instantaneu de combustibil în grame pe secundă, C_{sec} poate fi aproximat utilizând consumul specific de combustibil-*BSFC*-Brake Specific Fuel Consumption, exprimat în g/kWh:

$$C_{\text{sec}} = \frac{BSFC \cdot P_m}{3600 \cdot 1000}.$$

În regim staționar (viteză zero în trafic), motorul consumă combustibil doar pentru menținerea mișcării interne de rotație. Acest consum de bază, C_{id} , depinde direct de capacitatea cilindrică și temperatura motorului, fiind integrat ca valoare constantă pe secundă în perioadele de oprire completă.

4.3 Modelarea factorului de trafic

Pentru a transpune ecuațiile fizice într-un model aplicabil în trafic dinamic, introducem un coeficient multiplicativ de trafic, numit **Factor de congestie**, γ_{trafic} . Acest factor penalizează consumul teoretic ideal pe baza indicelui de aglomerație al segmentului:

$$C_{\text{real}} = C_{\text{teoretic}} \cdot (1 + \gamma_{\text{trafic}})$$

Termenul γ_{trafic} este calculat ca o funcție neliniară dependentă de raportul dintre viteza maximă legală a segmentului și viteza reală de deplasare permisă de trafic la aceea oră. Cu cât viteza medie scade sub pragul optim de eficiență al treptelor superioare de viteză, cu atât coeficientul crește, simulând pierderile masive de energie cauzate de frânările și accelerările succesive.

5 Arhitectura sistemului și implementarea numerică

În figura ce urmează, este oferită o imagine de ansamblu cu privire la structura aplicației create:

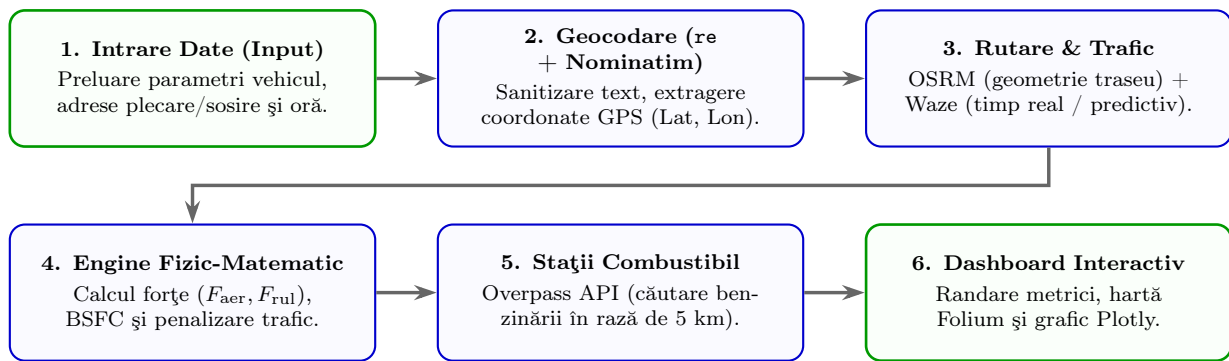


Figura 2: Schema logică a construcției aplicației OptiFuel.

5.1 Structura datelor utilizate

Pentru a asigura o implementare programatică flexibilă și ușor de extins, modelul va fi conceput utilizând o arhitectură de date modulară. Datele organizate în entități independente, vor permite modificarea rapidă a parametrilor fără a afecta logica centrală a algoritmului. Cele trei entități fundamentale interconectate sunt definite astfel:

- **Entitatea Vehicul:** Această structură stochează toate constantele fizice și tehnice specifice mașinii simulate. Proprietățile esențiale incluse sunt masa totală (m), tipul de combustibil (benzină/diesel) pentru determinarea densității, capacitatea cilindrică, coeficientul aerodinamic (C_x), aria frontală (A), consumul specific de combustibil ($BSFC$) și consumul la ralanti (C_{id}).
- **Entitatea Segment:** Reprezintă unitatea spațială de bază a traseului. Fiecare segment este caracterizat prin lungime (L), limită de viteză legală (v_{max}) și

înclinație medie (α), permițând calculul precis al forțelor de rezistență la fiecare pas al simulării.

- **Entitatea Profil Trafic:** Această entitate asociază fiecărui segment o matrice de comportament dinamic în funcție de timp. Aceasta conține viteza medie reală de deplasare (v_{real}) și numărul estimat de opriri complete pentru intervale orare predefinite: ore de vârf, ore de tranzit și ore libere (noapte).

Pentru o vizualizare clară a modului în care aceste entități interacționează în cadrul sistemului, structura acestor atribute este centralizată astfel:

Entitate	Atribute Principale	Rol în Modelul de Calcul
Vehicul	ID, m (kg), C_x , A (m^2), $BSFC$ (g/kWh), C_{id} (1/s), Combustibil	Calculul forțelor opozante, al puterii necesare și al consumului masic.
Segment	ID, Lungime (L), Limită viteză (v_{max}), Panta (α)	Determinarea rezistenței la pantă și a limitărilor fizice de viteză.
Profil Trafic	SegmentID, Interval orar, v_{real} (m/s), Probabilitate opriri	Ajustarea vitezei de rulare și calculul timpului petrecut la ralanti.

5.2 Logica algoritmului de calcul

Algoritmul va executa o simulare secvențială, parcurgând traseul segment cu segment și actualizând starea sistemului în timp real. Această abordare permite adaptarea dinamică a consumului pe măsură ce ora de deplasare se modifică pe parcursul călătoriei. Execuția algoritmului este următoarea:

1. **Inițializare:** În această primă etapă, programul încarcă în memorie instanța vehiculului selectat și vectorul de segmente ce alcătuiesc traseul. De asemenea, se definește ceasul global al simulării cu ora de plecare specificată de utilizator.
2. **Calcul dinamic pe segmente:** Pentru fiecare segment din structura traseului, algoritmul execută următoarele sub-operații:
 - *Interogarea profilului de trafic:* În funcție de valoarea curentă a ceasului global, se determină intervalul orar activ (vârf, zi sau noapte). Din entitatea **Profil Trafic** asociată segmentului se extrag viteza reală de deplasare (v_{real}) și numărul estimat de opriri complete (N_{opri}).

- *Calculul forțelor și al puterii la roată:* Utilizând ecuațiile fizice stabilite, se calculează forța totală de rezistență (F_{total}) necesară pentru a menține viteza v_{real} pe panta α . Puterea mecanică necesară la roată (P_r) este derivată direct ca produs între F_{total} și v_{real} .
 - *Determinarea consumului de bază:* Se calculează consumul teoretic necesar parcurgerii segmentului la viteză constantă, aplicând consumul specific ($BSFC$) la puterea motorului (P_m), raportată la randamentul transmisiei.
 - *Aplicarea penalizărilor de trafic:* Se calculează *Factorul de Congestie* (γ_{trafic}) ca o funcție neliniară de degradare a vitezei. Consumul de bază este multiplicat cu acest factor pentru a simula accelerările specifice traficului intens. Suplimentar, se adaugă penalizarea pentru timpul petrecut la ralanti ($N_{\text{oprire}} \cdot C_{\text{id}} \cdot T_{\text{stat}}$).
3. **Actualizare timp:** După parcurgerea fiecărui segment, timpul total de călătorie este incrementat cu durata de tranzit pe segment (L/v_{real}), plus timpul adițional cumulat din opririle la semafoare sau intersecții. Ora globală de plecare este actualizată corespunzător, asigurând selectarea corectă a ferestrelor de trafic pentru următoarele porțiuni de drum.
4. **Centralizare rezultate:** La finalizarea traseului, algoritmul însumează consumurile calculate pe fiecare segment pentru a obține consumul total exprimat în litri. Folosind lungimea totală a drumului parcurs, se calculează indicatorul standard de eficiență: consumul mediu exprimat în litri la 100 de kilometri (l/100 km).

5.3 Biblioteci utilizate în Python

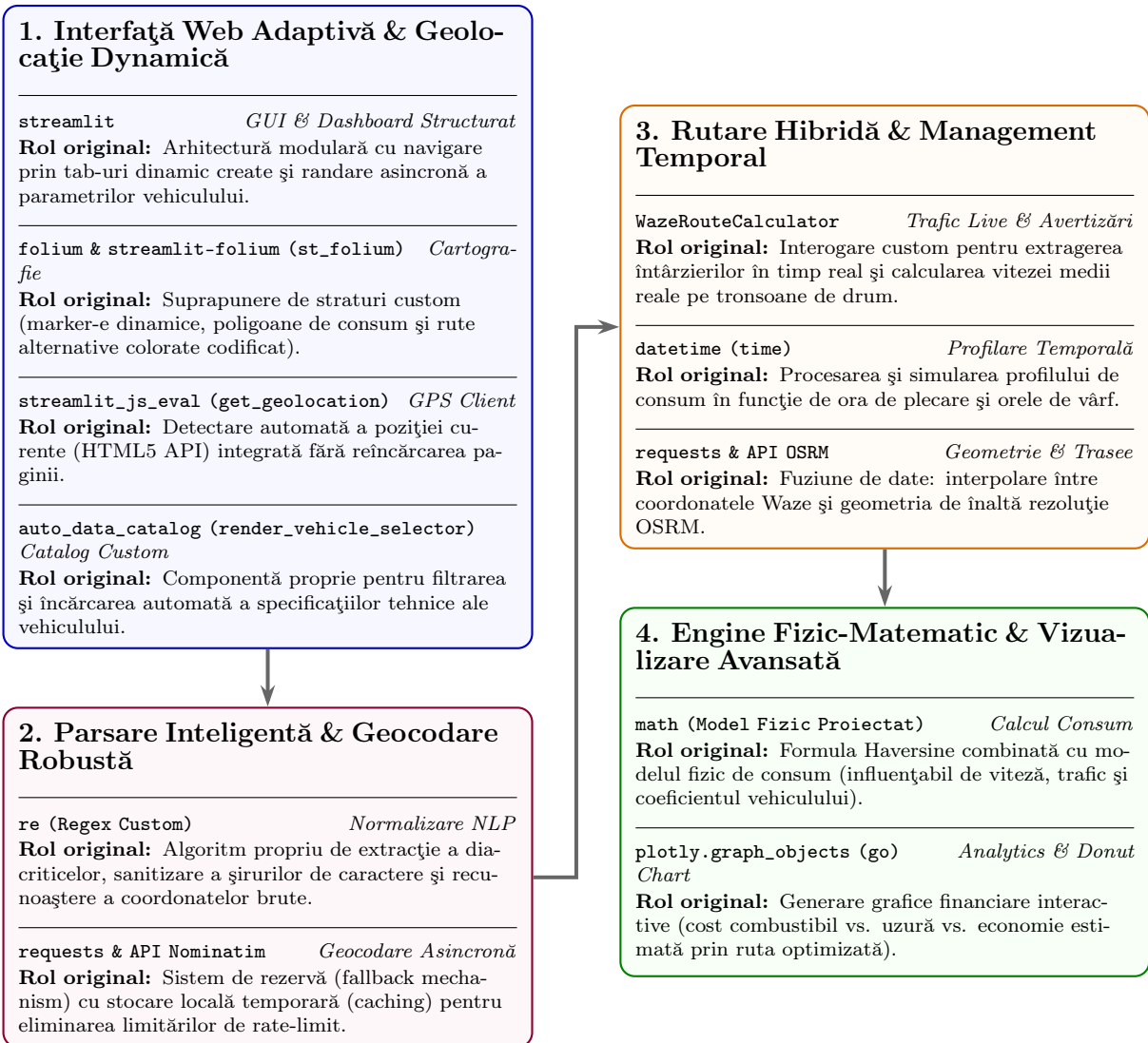


Figura 3: Bibliotecile utilizate în cadrul dezvoltării aplicației Python.

5.4 Descrierea algoritmilor și a modulelor de calcul

Algorithm 1 Geocoding Extraction Algorithm - DT

STEP 1: Input Cleaning and GPS Coordinate Check

```
1: Strip leading and trailing whitespace from location_name to obtain raw_text.
2: if raw_text matches coordinate pattern (e.g., "45.15170, 26.82130") then
3:   Parse latitude and longitude components from string.
4:   return (float(lat), float(lon))
5: end if
6: Remove street number prefixes ("nr.", "numarul") using case-insensitive regex.
7: Normalize multiple spaces into a single space to construct clean_text.
```

STEP 2: Fallback Query Generation

```
8: Initialize query list: queries_to_try = [clean_text].
9: if "romania" is not present in clean_text then
10:   Append clean_text + ", Romania" to queries_to_try.
11: end if
12: Remove numeric digits from clean_text to obtain without_number.
13: if without_number ≠ clean_text then
14:   Append without_number + ", Romania" to queries_to_try.
15: end if
16: Extract first term before comma to get first_term.
17: Append first_term + ", Romania" to queries_to_try.
```

STEP 3: API Interrogation and Coordinate Extraction

```
18: for each query in queries_to_try do
19:   try:
20:     Send HTTP GET request to OpenStreetMap Nominatim API with parameters:
21:       q = query, format = "json", limit = 1, timeout = 10s
22:     Parse HTTP response into JSON object res.
23:     if res contains valid result then
24:       return (float(res[0].lat), float(res[0].lon))
25:     end if
26:     except Exception:
27:       Continue to next query variant (ignore network/timeout errors).
28: end for
29: return (None, None)
```

Rol: Transformarea unui text introdus de utilizator (ex: "București") în coordonate GPS exacte (latitudine și longitudine). Funcția curăță textul de termeni inutili. De asemenea, asigură gestionarea cazurilor în care primește direct coordonate numerice.

Algorithm 2 Geometry Extraction Algorithm - DT

STEP 1: OSRM API Request Execution

```
1: try:
2:   Construct OSRM endpoint URL using coordinates  $(lon\_p, lat\_p)$  and  $(lon\_s, lat\_s)$ .
3:   Send HTTP GET request with custom User-Agent header and timeout = 10s.
4: if response.status_code  $\neq$  200 then
5:   Log API rejection error message.
6:   return (None, None, [], [])
7: end if
```

STEP 2: Route Geometry and Distance Parsing

```
8:   Parse HTTP response into JSON object res.
9: if res contains key "routes" and  $len(res["routes"]) > 0$  then
10:   Extract primary route  $route \leftarrow res["routes"][0]$ .
11:   Convert distance:  $dist\_km \leftarrow route.distance/1000.0$ .
12:   Convert duration:  $durata\_min \leftarrow route.duration/60.0$ .
13:   Invert coordinate pairs to  $[lat, lon]$  format for mapping (coords).
```

STEP 3: Segment Extraction and Return

```
14:   Initialize segment list segmente  $\leftarrow []$ .
15:   for each step in route.legs[0].steps do
16:     Extract step attributes: distance, duration, and name.
17:     Append structured segment dictionary to segmente.
18:   end for
19:   return ( $dist\_km, durata\_min, coords, segmente$ )
20: end if
21: except Exception as e:
22:   Log technical exception error e.
23: return (None, None, [], [])
```

Rol: Preia coordonatele de plecare și sosire și le trimite către serverele OSRM pentru a construi traseul rutier. Returnează distanța totală, timpul ideal, geometria traseului - punctele necesare pentru a desena linia pe hartă și împarte drumul în segmente mai mici pentru a putea calcula consumul pas cu pas.

Algorithm 3 Fuel Consumption Algorithm - DT

STEP 1: Edge Case Checking and Vehicle Parameters
1: **if** *distanta_m* == 0 **or** *durata_s* == 0 **then**
2: **return** 0.0
3: **end if**
4: Extract mass $m \leftarrow \text{float}(\text{spec}["\text{masa}"])$ and drag coefficient $Cx \leftarrow \text{spec.get}("Cx", 0.32)$.
5: **if** "*latime*" $\in$ *spec* **and** "*inaltime*" $\in$ *spec* **then**
6: Frontal area $A \leftarrow (\text{width}_m \times \text{height}_m) \times 0.81$
7: **else**
8: $A \leftarrow \text{spec.get}("A", 2.15)$
9: **end if**
10: Extract generation year *an_generatie* via Regex matching (default 2015).

STEP 2: BSFC and Drivetrain Efficiency Estimation
11: **if** "*benzin*" $\in$ *combustibil* **then**
12: **if** *an_generatie* < 2010 **then**
13: $\text{bsfc_estimat} \leftarrow 280 \text{ g/kWh}$, $\eta_{tr} \leftarrow 0.86$
14: **else**
15: $\text{bsfc_estimat} \leftarrow 250 \text{ g/kWh}$, $\eta_{tr} \leftarrow 0.90$
16: **end if**
17: **else** ▷ Diesel Engine
18: **if** *an_generatie* < 2010 **then**
19: $\text{bsfc_estimat} \leftarrow 220 \text{ g/kWh}$, $\eta_{tr} \leftarrow 0.88$
20: **else**
21: $\text{bsfc_estimat} \leftarrow 205 \text{ g/kWh}$, $\eta_{tr} \leftarrow 0.90$
22: **end if**
23: **end if**
24: Set physical constants: $\rho \leftarrow 1.225 \text{ kg/m}^3$, $g \leftarrow 9.81 \text{ m/s}^2$, $f \leftarrow 0.015$, $\alpha \leftarrow 0.0$.

STEP 3: Kinematics and Physical Forces Calculation
25: Adjust duration for traffic: $\text{durata_reala}_s \leftarrow \text{durata}_s \times \text{trafic_multiplier}$.
26: Average velocity $v \leftarrow \text{distanta}_m / \text{durata_reala}_s$.
27: Rolling resistance $F_{rul} \leftarrow m \cdot g \cdot f \cdot \cos(\alpha)$.
28: Aerodynamic drag $F_{aer} \leftarrow 0.5 \cdot \rho \cdot Cx \cdot A \cdot v^2$.
29: Acceleration force $F_{dem} \leftarrow (m \cdot 1.5 \cdot 0.1)$ if *opriri* > 0 else 0.
30: $F_{total} \leftarrow F_{rul} + F_{aer} + F_{panta} + F_{dem}$.
31: Wheel power $P_r \leftarrow F_{total} \cdot v$; Engine power $P_m \leftarrow P_r / \eta_{tr}$; $P_{m,kW} \leftarrow P_m / 1000.0$.

STEP 4: Fuel Consumption Integration and Return
32: Density $\rho_{fuel} \leftarrow 0.74 \text{ kg/L}$ (Gasoline) or 0.83 kg/L (Diesel).
33: Fuel rate in motion: $g/s \leftarrow (BSFC \cdot P_{m,kW}) / 3600.0$.
34: Volume in motion: $\text{consum_miscare}_L \leftarrow (g/s / (\rho_{fuel} \cdot 1000)) \times \text{durata_reala}_s$.
35: Idle loss: $\text{consum_stationare}_L \leftarrow (\text{opriri} \times 15s) \times C_{idle}$.
36: **return** $\text{consum_miscare}_L + \text{consum_stationare}_L$

Rol: Calculează exact câți litri de combustibil se ard pe o porțiune scurtă de drum. Face acest lucru aplicând formule fizice pentru forțele de rezistență și combinându-le cu specificațiile tehnice ale mașinii.

Algorithm 4 Live Traffic Algorithm - DT

STEP 1: Waze API Initialization and Live Route Calculation

```
1: try:  
2:   Construct string coordinates:  $start\_coords \leftarrow "lat\_p,lon\_p"$ ,  $end\_coords \leftarrow "lat\_s,lon\_s"$ .  
3:   Instantiate WazeRouteCalculator instance waze for region "EU".  
4:   Compute live route metrics:  $(route\_time, route\_distance) \leftarrow waze.calc\_route\_info()$ .
```

STEP 2: Traffic Scenario Adjustment (Critical Hours Logic)

```
5: if scenariu == "A" then                                     ▷ Peak hour traffic  
6:    $route\_time \leftarrow route\_time \times 1.35$   
7: else if scenariu == "C" then                                   ▷ Nighttime free-flow traffic  
8:    $route\_time \leftarrow route\_time \times 0.90$   
9: end if                                                         ▷ Default scenario "B" preserves unmodified Waze live travel time
```

STEP 3: Return Metrics and Exception Handling

```
10: return ( $route\_distance, route\_time$ )  
11: except Exception:  
12: return (None, None)
```

Rol: Se conectează la Waze pentru a extrage timpul și distanța reale, influențate de traficul live. În plus, această funcție oferă posibilitatea utilizatorului să aleagă scenariul în care face deplasarea (trafic de vârf, trafic normal sau trafic nocturn).

6 Dezvoltarea interfeței grafice și arhitectura aplicației

6.1 Cadrul Streamlit și modelul de execuție

Dezvoltarea interfeței grafice se bazează pe cadrul Streamlit, utilizând opțiunea de afișare extinsă `wide` pentru o organizare optimă a spațiului vizual. Structura aplicației este divizată modular în coloane dinamice, folosind `st.columns`, care permite gruparea intuitivă a câmpurilor pentru adrese, dată, oră și a cardurilor de rezultate. Aspectul estetic al acestor carduri este personalizat prin utilizarea unui bloc CSS dedicat, care definește stilul vizual, culorile și dimensiunile fonturilor. Deoarece modelul de execuție al Streamlit re-evaluează scriptul la fiecare interacțiune cu interfața, algoritmi complecși de rutare și calcul fizic sunt izolați în spatele unui buton declanșator, prevenind astfel apelurile inutile către serviciile externe.

6.2 Modulele interactive de vizualizare

Vizualizarea spațială a traseului se realizează prin integrarea modului `st.folium`, care randează o hartă interactivă centrată automat pe mijlocul geografic al rutei. Tra-

seul este marcat printr-o linie poligonală albastră, iar punctele de plecare și sosire sunt evidențiate prin markere specifice.

6.3 Structura bazei de date auto

Modulul `auto.data.catalog` gestionează preluarea specificațiilor tehnice ale vehiculelor. Interfața oferă un sistem de selecție în cascadă care ghidează utilizatorul de la marcă și model până la generație și motorizare. Parametrii fizici, precum masa, cilindreea, puterea și consumurile declarate, sunt extrași automat prin expresiile regulate, iar consumurile furnizate ca interval sunt mediate aritmetic. (de reformulat) Totodată, utilizatorul are posibilitatea de a ajusta în timp real încărcătura suplimentară sau de a personaliza consumul mixt utilizat în ecuațiile de calcul.

7 Studiu de caz și rezultate experimentale

7.1 Definirea scenariilor de test

Pentru a evalua acuratețea și sensibilitatea modelului dezvoltat, s-a definit un traseu urban-reprezentativ cu o lungime de 12 km, împărțit în 15 segmente cu profile și limite de viteză variabile. Scenariul presupune parcurgerea acestui traseu în trei momente temporale distincte, caracterizate prin comportamente de trafic diferite:

Scenariu	Tip de Trafic	Viteză Medie	Regim de Rulare și Opriri
Scenariul A (Ora 08:15)	Trafic de vârf de dimineață.	12 – 15 km/h.	Regim <i>Stop-and-Go</i> sever, frecvență maximă a opririlor la semafoare.
Scenariul B (Ora 14:30)	Trafic urban normal de zi.	28 – 32 km/h.	Rulare urbană standard, cu opriri moderate.
Scenariul C (Ora 23:30)	Trafic nocturn liber.	45 – 50 km/h.	Viteză de croazieră stabilă, fără opriri majore sau congestii rutiere.

7.2 Compararea rezultatelor pe tipuri de motorizări

Simulările au fost rulate pentru două vehicule cu profiluri tehnice fundamentale diferite:

—————Rezultate simulare—————

Vehicul	Masă	Cilindree	Combustibil	Transmisie
Vehiculul 1 (Clasa Subcompactă)	1100 kg	1.2 litri (aspirat)	Benzină	Manuală
Vehiculul 2 (Clasa SUV Mediu)	1800 kg	2.0 litri (supraalimentat)	Diesel	Automată

7.3 Interpretarea grafică a datelor

—————Realizare grafice pentru rezultate simulare și interpretarea lor—————

8 Resurse

Pentru dezvoltarea, testarea și calibrarea modelului numeric de simulare a consumului de combustibil, a fost necesară utilizarea unui set de resurse tehnice, împărțite în componente hardware, instrumente software și surse de date.

Sinteza acestor resurse este structurată în tabelul de mai jos și detaliată în continuare:

- **Resurse Hardware:** Algoritmul implementat nu necesită putere de calcul masivă (cum ar fi plăci grafice dedicate), rulând eficient pe orice sistem de calcul personal modern capabil să execute interpretorul Python standard.
- **Resurse Software:** S-a optat exclusiv pentru instrumente *open-source* (Python, biblioteci standard precum `math`) și platforme bazate pe cloud (Overleaf, Google Colab), reducând la zero costurile de licențiere pentru realizarea proiectului.
- **Resurse de Date:** Datele privind coeficienții aerodinamici (C_x), masele vehiculelor și hărțile de consum ($BSFC$) au fost preluate din literatura de specialitate și din fișele tehnice oficiale ale produselor analizate, asigurând o precizie ridicată a simulării.

Tip Resursă	Denumire / Specificație	Rol în Proiect
Hardware	Laptop/PC de lucru (recomandat min. 8 GB RAM, procesor Dual-Core).	Rularea mediului de dezvoltare și a simulărilor numerice.
Software (Core)	Python (versiunea 3.10 sau mai nouă).	Limbajul de bază pentru scrierea algoritmului de calcul.
Software (Auxiliar)	Editor de cod (Visual Studio Code / PyCharm sau Google Colab).	Scrierea, depanarea (debugging) și testarea codului Python.
Redactare	Platforma LaTeX (Overleaf).	Tehnoredactarea și formatarea documentației academice.
Date	Fîșe tehnice auto (BSFC, mase) și estimări de trafic urban.	Alimentarea bazei de date cu valori reale pentru studii de caz.

9 Concluzii și perspective de dezvoltare?

Bibliografie

- [1] Trifan Darko (2026). *Practică CITI 2026*, UNSTPB, București, <https://github.com/darkotei/Practica-CITI-2026>
- [2] Untaru, M., et al. (1982). *Dinamica autovehiculelor*. Editura Didactică și Pedagogică, București.
- [3] Gillespie, T. D. (1992). *Fundamentals of Vehicle Dynamics*. Society of Automotive Engineers (SAE).
- [4] Heywood, J. B. (1988). *Internal Combustion Engine Fundamentals*. McGraw-Hill Education.
- [5] Bosch, R. (2014). *Bosch Automotive Handbook* (9th Edition). Wiley.
- [6] Akçelik, R. (1989). *Efficiency and Fuel Consumption Models for Traffic Signal Design*. Australian Road Research.
- [7] Treiber, M., Kesting, A. (2013). *Traffic Flow Dynamics: Data, Signal Processing, and Theory*. Springer.
- [8] Waze Developers (2026). *Waze Route Calculator Python Module & API Documentation*.
- [9] OSRM Team (2026). *Open Source Routing Machine (OSRM) API Documentation*.
- [10] OpenStreetMap Contributors (2026). *Nominatim Geocoding API*.
- [11] Streamlit Inc. (2026). *Streamlit Documentation*. <https://docs.streamlit.io>
- [12] Folium Contributors (2026). *Folium: Python Data, Leaflet.js Maps*. <https://python-visualization.github.io/folium>
- [13] Plotly Technologies Inc. (2026). *Plotly Graphing Libraries*. <https://plotly.com/python/>
- [14] Python Software Foundation (2026). *The Python Standard Library: math module*. <https://docs.python.org/3/library/math.html>